

LAPORAN PRAKTIKUM WEB PROGRAMMING
MODUL 05



Nama : Faril Isra Albiansyah
NIM : 240605110087
Kelas : WEB Programming B
Tanggal : 09 – 03 – 2026

JURUSAN TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM
MALANG
GENAP 2026

I. Tujuan

Berdasarkan modul praktikum Bab 5, mahasiswa diharapkan memiliki kemampuan untuk:

- Memahami Fungsi Form: Menjelaskan konsep dasar form HTML sebagai sarana utama untuk menerima dan mengumpulkan input data dari pengguna pada aplikasi web.
- Membedakan Metode Pengiriman: Memahami perbedaan mendasar antara metode GET (data terlihat di URL) dan POST (data tersembunyi) dalam proses pengiriman data ke server.
- Penggunaan Variabel Superglobal: Mampu mengambil, menampung, dan memproses data yang dikirimkan dari form menggunakan PHP melalui variabel superglobal, khususnya `$_POST`.
- Implementasi Validasi Data: Menerapkan teknik validasi sederhana (baik di sisi klien maupun server) untuk memastikan bahwa data yang dikirim tidak kosong dan sesuai dengan kebutuhan aplikasi.
- Pemrosesan Asynchronous: Membangun simulasi aplikasi kontak sederhana yang mampu memproses, menyimpan ke database, dan menampilkan kembali data ke tabel secara instan (tanpa *refresh* halaman) menggunakan kombinasi PHP, JavaScript (AJAX/Fetch), dan format JSON

II. Langkah Kerja

Langkah-langkah berikut dilakukan secara sistematis untuk membangun aplikasi kontak sederhana yang memproses data dari sisi klien hingga ke server database:

1. Perancangan Antarmuka Form HTML

- Membuat struktur form HTML menggunakan elemen `<form>`, `<input>` (text, email), dan `<textarea>` untuk menerima data nama, email, dan pesan.
- Mengatur metode pengiriman form menggunakan `method="post"` agar pengiriman data lebih aman dan tidak terlihat pada baris alamat (URL).

- Menerapkan validasi awal di sisi klien menggunakan atribut `required` pada setiap elemen input untuk mencegah pengiriman form yang kosong.

2. Penangkapan Data di Sisi Server (PHP)

- Membuat file PHP (misalnya `proses_kontak.php`) untuk menampung data yang dikirimkan.
- Menggunakan variabel superglobal `$_POST` untuk menangkap data berdasarkan atribut `name` yang telah ditetapkan pada elemen input form HTML (contoh: `$_POST['nama']`).

3. Koneksi dan Penyimpanan Data ke Database

- Membuat script terpisah (misalnya `koneksi.php`) untuk membangun jalur koneksi antara aplikasi web dengan server database MySQL menggunakan perintah `mysqli_connect`.
- Melakukan validasi tambahan di sisi server menggunakan perintah `!empty()` untuk memastikan data benar-benar memiliki nilai sebelum diproses.
- Menjalankan perintah SQL `INSERT INTO` untuk menyimpan data nama, email, dan pesan secara permanen ke dalam tabel database.

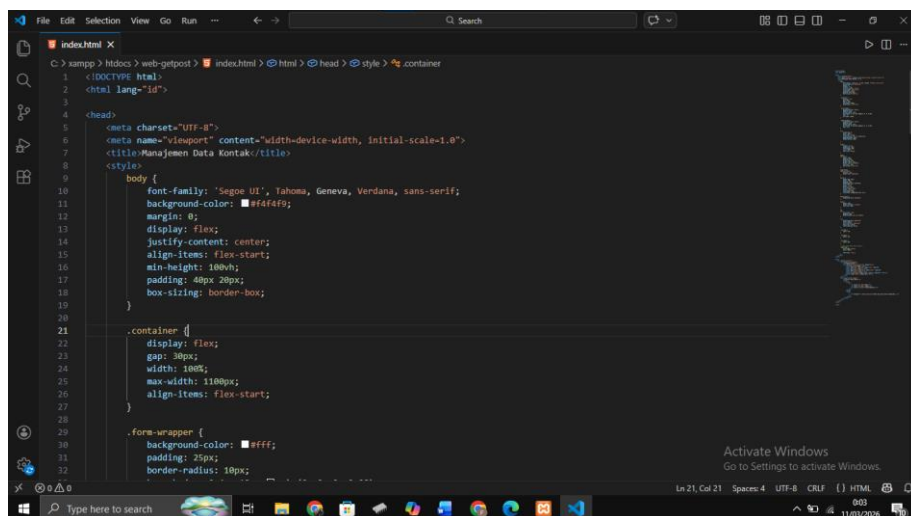
4. Implementasi AJAX untuk Tampilan Asynchronous

- Modifikasi Form HTML: Menghapus atribut `action` dan `method` pada tag form, lalu menggantinya dengan atribut `id="formKontak"` agar pengiriman data ditangani sepenuhnya oleh JavaScript.
- Pengiriman Data (Fetch API): Menggunakan JavaScript dengan fungsi `fetch()` untuk mengirimkan data formulir (`FormData`) ke file PHP pemroses di latar belakang.
- Modifikasi Respons PHP (JSON): Mengubah format keluaran pada file `proses_kontak.php` menggunakan fungsi `json_encode()`. Server akan merespons dengan mengirimkan paket objek JSON yang berisi status keberhasilan beserta data yang baru saja divalidasi.
- Manipulasi DOM: Menangkap paket data JSON pada sisi klien, kemudian menggunakan JavaScript (metode `insertRow(0)`) untuk menyisipkan data

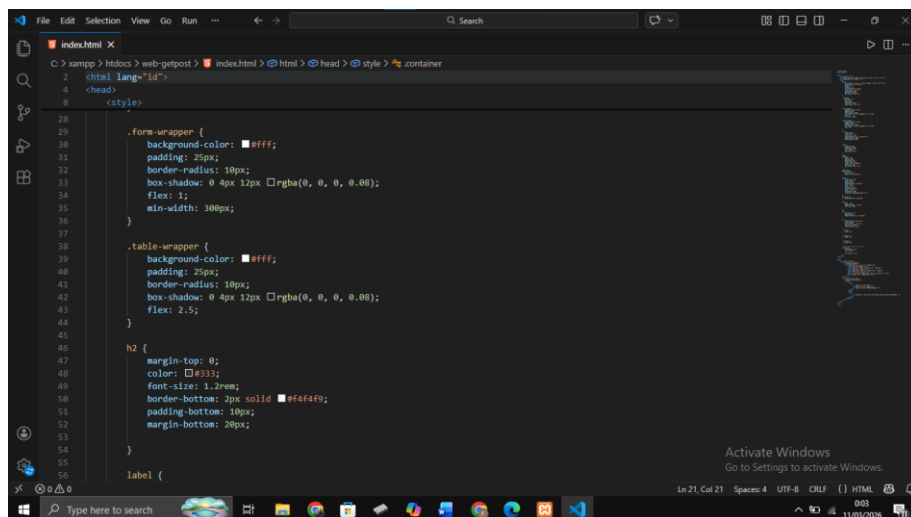
tersebut sebagai baris baru di posisi teratas tabel HTML secara instan, tanpa perlu memuat ulang (*refresh*) halaman.

III. Screenshot Hasil

1. Membuat Form HTML Sederhana



```
1 <!DOCTYPE html>
2 <html lang="id">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Manajemen Data Kontak</title>
8   <style>
9     body {
10       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
11       background-color: #f4f4f9;
12       margin: 0;
13       display: flex;
14       justify-content: center;
15       align-items: flex-start;
16       min-height: 100vh;
17       padding: 40px 20px;
18       box-sizing: border-box;
19     }
20
21     .container {
22       display: flex;
23       gap: 30px;
24       width: 100%;
25       max-width: 1180px;
26       align-items: flex-start;
27     }
28
29     .form-wrapper {
30       background-color: #fff;
31       padding: 25px;
32       border-radius: 10px;
```

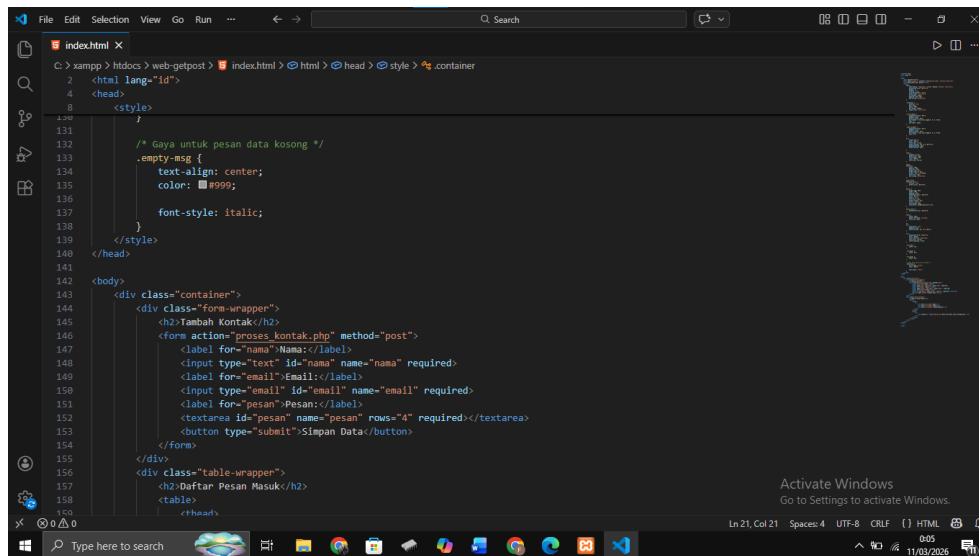


```
33     }
34
35     .table-wrapper {
36       background-color: #fff;
37       padding: 25px;
38       border-radius: 10px;
39       box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
40       flex: 2.5;
41     }
42
43     h2 {
44       margin-top: 0;
45       color: #333;
46       font-size: 1.2rem;
47       border-bottom: 2px solid #f4f4f9;
48       padding-bottom: 10px;
49       margin-bottom: 20px;
50     }
51
52     label {
```

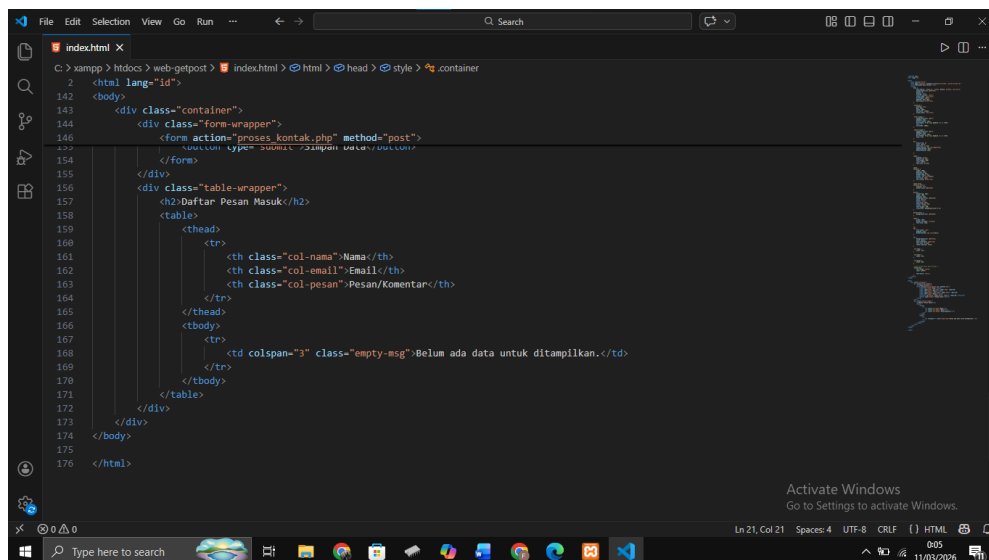
```
File Edit Selection View Go Run -- Search
index.html X
C:\xampp\htdocs> web-getpost > index.html > html > head > style > .container
2 <html lang="id">
4 <head>
8 <style>
54 }
55
56 label {
57     display: block;
58     margin-top: 15px;
59     font-weight: 600;
60     color: #555;
61     font-size: 0.9em;
62 }
63
64 input,
65 textarea {
66     width: 100%;
67     padding: 10px;
68     margin-top: 5px;
69     border-radius: 5px;
70     border: 1px solid #ddd;
71     font-size: 14px;
72     box-sizing: border-box;
73 }
74
75 input:focus,
76 textarea:focus {
77     outline: none;
78     border-color: #4CAF50;
79 }
80
81 button {
82     margin-top: 20px;
```

```
File Edit Selection View Go Run -- Search
index.html X
C:\xampp\htdocs> web-getpost > index.html > html > head > style > .container
2 <html lang="id">
4 <head>
8 <style>
80 }
81
82 button {
83     margin-top: 20px;
84     width: 100%;
85     padding: 12px;
86     background-color: #4CAF50;
87     color: white;
88     border: none;
89     font-size: 15px;
90     border-radius: 5px;
91     cursor: pointer;
92     font-weight: 600;
93     transition: background-color 0.3s;
94 }
95
96 button:hover {
97     background-color: #45a049;
98 }
99
100 table {
101     width: 100%;
102     border-collapse: collapse;
103     font-size: 14px;
104 }
105
106 th,
107 td {
108     text-align: left;
109     padding: 12px;
110     border-bottom: 1px solid #eee;
```

```
File Edit Selection View Go Run -- Search
index.html X
C:\xampp\htdocs> web-getpost > index.html > html > head > style > .container
2 <html lang="id">
4 <head>
8 <style>
104 th,
105 td {
106     text-align: left;
107     padding: 12px;
108     border-bottom: 1px solid #eee;
109 }
110
111 th {
112     background-color: #f4f4f4;
113     color: #666;
114     text-transform: uppercase;
115     font-size: 0.9em;
116     letter-spacing: 0.5px;
117 }
118
119 .col-nama {
120     width: 35%;
121 }
122
123 .col-email {
124     width: 25%;
125 }
126
127 .col-pesan {
128     width: 60%;
129 }
130
131 /* Gaya untuk pesan data kosong */
132
```



```
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```



```
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Membangun struktur HTML sebagai penerima data dari pengguna.

Formulir disusun menggunakan elemen dasar text, email, dan textarea dengan desain sederhana. Selanjutnya, dengan metode GET atau POST, data tersebut akan dikirim ke server untuk divalidasi dan disimpan yang akan ditunjukkan pada tahap berikutnya.

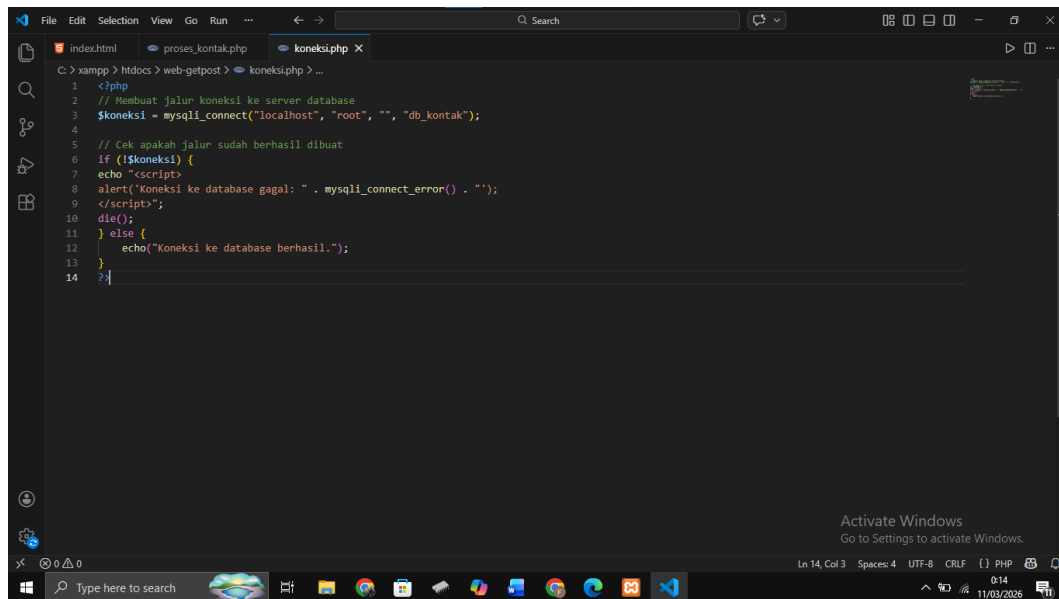
a. Mekanisme Pengiriman Data: Formulir menggunakan metode POST agar data yang diinputkan pengguna dikirim ke server secara tersembunyi. Berbeda dengan metode GET, data pada Metode POST tidak akan terlihat pada baris alamat (address bar) web browser, sehingga lebih rapi dan aman untuk pengiriman data.

- b. Validasi Sisi Klien: Setiap elemen input dilengkapi dengan atribut required. Validasi awal disisi klien ini memastikan bahwa pengguna tidak dapat mengirimkan formulir sebelum seluruh kolom teks, email, dan pesan terisi.
- c. Tujuan Pemrosesan Data: Atribut action pada formulir diarahkan ke file proses_kontak.php. File tersebut berfungsi sebagai penampung dan pengolah data sebelum disimpan ke dalam database.
- d. Estetika dan Interaktivitas: Antarmuka (GUI) didesain dengan pendekatan bersih (clean) dan responsif menggunakan wrapper di posisi tengah.

Output :

The screenshot displays a web application interface. On the left, the 'Tambah Kontak' form includes three input fields for 'Nama:', 'Email:', and 'Pesan:', each marked as required. A green 'Simpan Data' button is positioned at the bottom of this form. On the right, the 'Daftar Pesan Masuk' section features a table with columns for 'NAMA', 'EMAIL', and 'PESAN/KOMENTAR'. The table is currently empty, and a message 'Belum ada data untuk ditampilkan.' is shown below it. The browser's address bar indicates the URL 'localhost/web-getpost/'. The Windows taskbar at the bottom shows the date as 11/03/2026 and the time as 0:05.

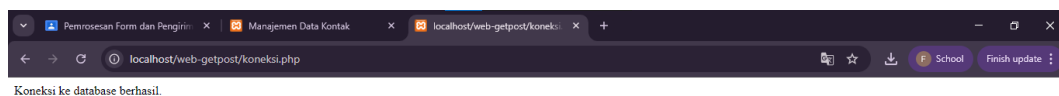
2. Menangkap Data dengan Variabel Superglobal



```
1 <?php
2 // Membuat jalur koneksi ke server database
3 $koneksi = mysqli_connect("localhost", "root", "", "db_kontak");
4
5 // Cek apakah jalur sudah berhasil dibuat
6 if (!$koneksi) {
7     echo "<script>
8     alert('Koneksi ke database gagal: " . mysqli_connect_error() . "');
9     </script>";
10    die();
11 } else {
12     echo("Koneksi ke database berhasil.");
13 }
14 >?>
```

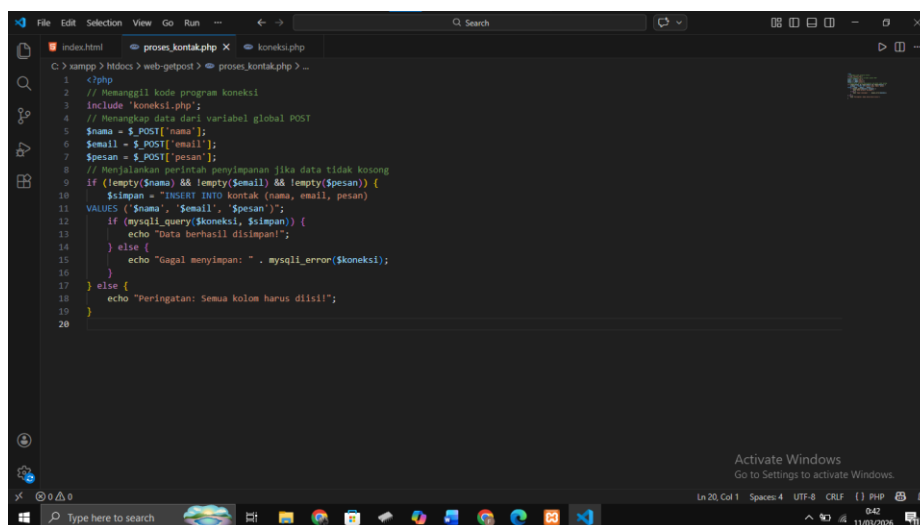
Setelah data tersimpan dalam variabel, langkah berikutnya adalah membuat koneksi agar data tersebut dapat disimpan secara permanen ke dalam database.

Output :



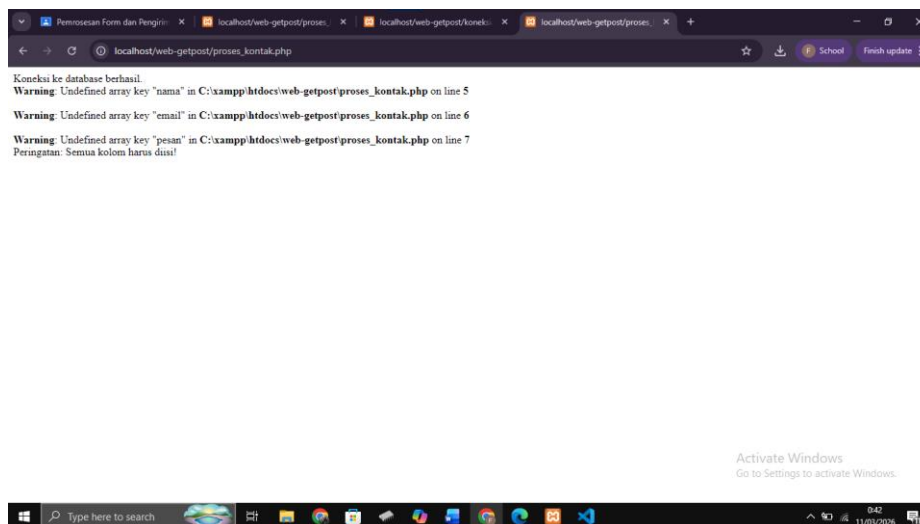
Kode program koneksi ini sebaiknya ditulis dalam file tersendiri, misalnya koneksi.php. Pemisahan ini dilakukan agar kode lebih rapi dan mudah dikelola. Selain itu, file tersebut dapat dipanggil kembali oleh halaman lain yang membutuhkan akses ke database tanpa perlu menulis ulang yang sama. Untuk

memberikan pengalaman pengguna yang lebih baik, pesan kesalahan koneksi dapat ditampilkan menggunakan jendela peringatan (alert) dari JavaScript dengan menyisipkan kode JavaScript dalam echo sebelum program dihentikan. Agar kode program dapat melakukan pengecekan kesalahan secara mandiri, laporan error otomatis dari sistem perlu dinonaktifkan terlebih dahulu. Setelah koneksi ke database berhasil dilaksanakan, berikutnya adalah proses penyimpanan data. Pada bagian ini, data yang diterima dari form akan disimpan ke database menggunakan perintah SQL.



```
1 <?php
2 // Mengambil kode program koneksi
3 include 'koneksi.php';
4 // Mengurus data dari variabel global POST
5 $nama = $_POST['nama'];
6 $email = $_POST['email'];
7 $pesan = $_POST['pesan'];
8 // Menjalankan perintah penyimpanan jika data tidak kosong
9 if (!empty($nama) && !empty($email) && !empty($pesan)) {
10     $simpan = "INSERT INTO kontak (nama, email, pesan)
11     VALUES ('$nama', '$email', '$pesan')";
12     if (mysqli_query($koneksi, $simpan)) {
13         echo "Data berhasil disimpan!";
14     } else {
15         echo "Gagal menyimpan: " . mysqli_error($koneksi);
16     }
17 } else {
18     echo "Peringatan: Semua kolom harus diisi!";
19 }
20
```

Output :



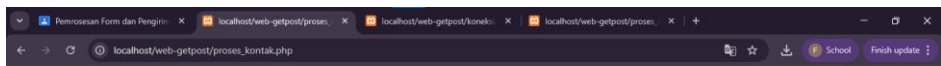
Dari kode program tersebut terlihat bahwa untuk menyimpan data ke database diawali dengan memanggil file koneksi.php agar sistem dapat terhubung ke server database. Selanjutnya, data yang dikirim melalui formulir ditangkap menggunakan variabel global `$_POST` untuk disimpan ke dalam variabel `$nama`, `$email`, serta `$pesan`. Dari kode program tersebut terlihat bahwa sebelum data disimpan, terdapat pengecekan kondisi untuk memastikan semua kolom input telah terisi. Jika data sudah lengkap, kode program akan menjalankan perintah SQL INSERT untuk memasukkan data ke dalam tabel kontak. Hasil akhirnya adalah pesan konfirmasi mengenai status penyimpanan data pada sistem. Untuk memastikan data dari formulir benar-benar telah tersimpan, pemeriksaan dapat dilakukan melalui phpMyAdmin dengan melihat isi tabel pada database tersebut. Karena pada tahap ini data yang baru saja tersimpan belum ditampilkan secara otomatis pada tabel di sebelah kanan formulir.

The screenshot shows a web browser window with the URL `localhost/web-getpost/`. The page contains two main sections:

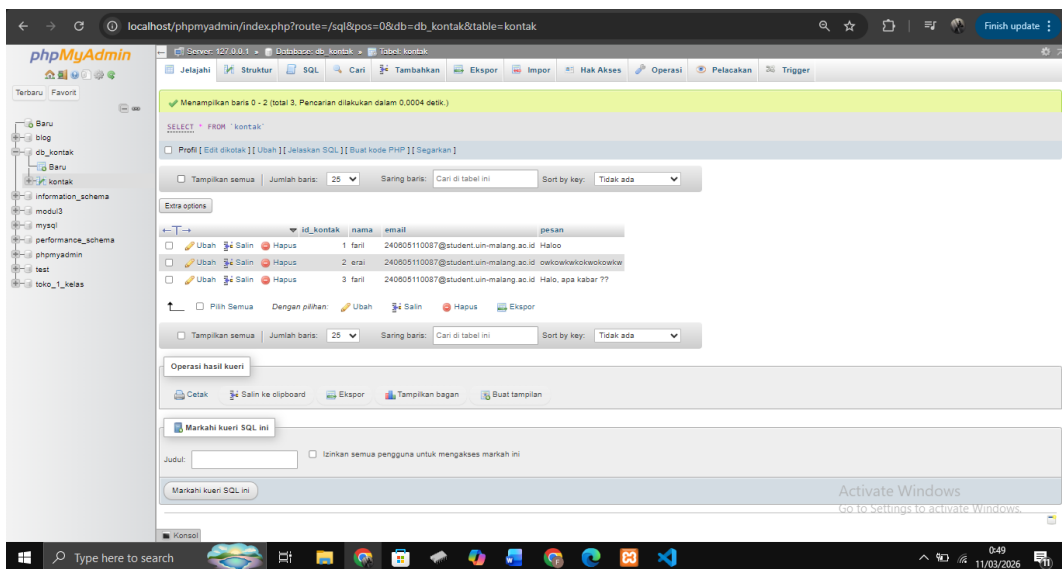
- Tambah Kontak**: A form with three input fields: "Nama:" (filled with "faril"), "Email:" (filled with "240605110087@student.uin-malang.ac.id"), and "Pesan:" (filled with "Halo, apa kabar ??"). Below the fields is a green button labeled "Simpan Data".
- Daftar Pesan Masuk**: A table with three columns: "NAMA", "EMAIL", and "PESAN/KOMENTAR". The table is currently empty, with a message below it stating "Belum ada data untuk ditampilkan".

The browser's taskbar at the bottom shows the date and time as 043 11/05/2026.

Jika disubmit melalui form manajemen kontak



Jika dilihat di database :



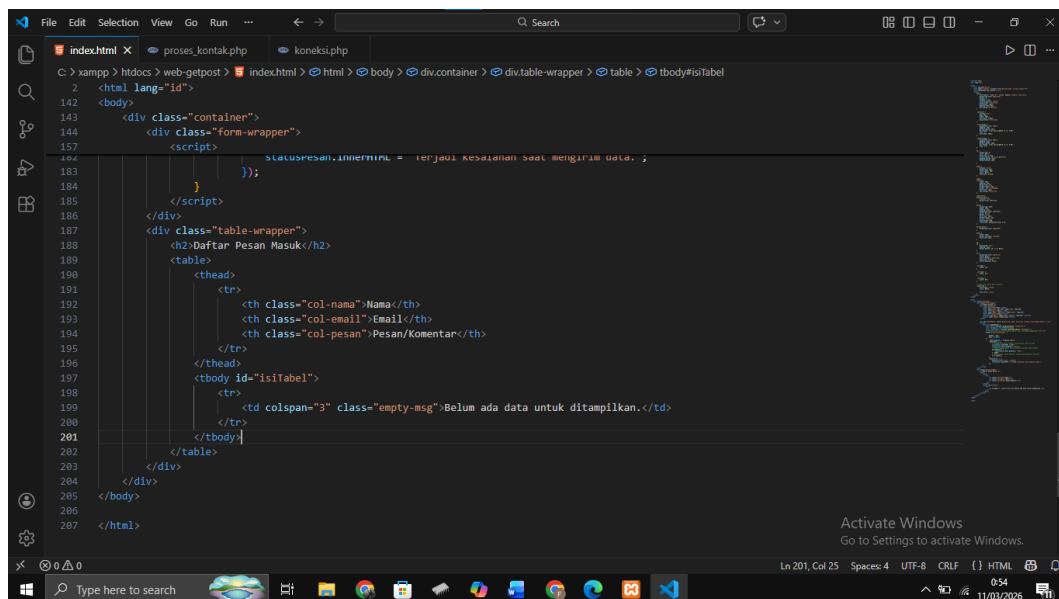
3. Menampilkan Data ke Tabel Secara Asynchronous

Pada tahap ini, sistem akan dikembangkan agar data yang baru saja disimpan dapat langsung muncul pada tabel di sebelah kanan formulir tanpa perlu melakukan penyegaran (refresh) halaman. Teknik ini memanfaatkan AJAX dengan fungsi `fetch()` pada JavaScript untuk mengirim data ke server, serta format JSON sebagai media pertukaran datanya.

Untuk menampilkan data secara asynchronous diawali dengan mengubah respons dari file `proses_kontak.php` menjadi objek JSON. Setelah perintah SQL INSERT berhasil dijalankan, server akan mengirimkan balik data tersebut (nama, email, dan pesan) ke halaman formulir. JavaScript kemudian menangkap paket data tersebut dan menyisipkannya sebagai baris baru ke dalam elemen `<tbody>` pada tabel secara otomatis. Namun sebelumnya, akan dibahas terlebih dahulu cara menampilkan pesan konfirmasi penyimpanan data menggunakan AJAX. Dengan teknik ini, pesan “Data berhasil disimpan!” dapat muncul secara instan tepat di bawah tombol tanpa harus memuat ulang halaman maupun berpindah ke halaman proses. Pendekatan ini menjadi langkah awal untuk memahami bagaimana JavaScript berkomunikasi dengan PHP di latar belakang. Setelah pesan berhasil dimunculkan, barulah logika tersebut dikembangkan lebih lanjut untuk menyisipkan data yang baru saja tersimpan ke dalam table secara otomatis. Langkah pertama adalah melakukan perubahan pada file tag `<form>` dengan menghapus atribut `action` dan `method`, karena proses pengiriman data kini sepenuhnya ditangani oleh JavaScript. Sebagai gantinya, tambahkan atribut `id="formKontak"` agar formulir dapat dikenali oleh skrip. Selanjutnya, tambahkan sebuah elemen `<div>` kosong di bawah tag penutup `</form>` sebagai wadah untuk menampilkan pesan konfirmasi.

```
index.html X proses_kontak.php koneksi.php
C:\xampp\htdocs> web-getpost > index.html > html > body > div.container > div.form-wrapper > form#formKontak > button
2 <html lang="id">
4 <head>
140 </head>
141
142 <body>
143 <div class="container">
144 <div class="form-wrapper">
145 <h2>Tambah Kontak</h2>
146 <form id="formKontak">
147 <label for="nama">Nama:</label>
148 <input type="text" id="nama" name="nama" required>
149 <label for="email">Email:</label>
150 <input type="email" id="email" name="email" required>
151 <label for="pesan">Pesan:</label>
152 <textarea id="pesan" name="pesan" rows="4" required></textarea>
153 <button type="button" onclick="simpanData()">Simpan Data</button>
154 </form>
155
156 <div id="statusPesan" style="margin-top: 15px; font-size: 0.9rem; text-align:center;"></div>
157 <script>
158 function simpanData() {
159 const form = document.getElementById('formKontak');
160 const formData = new FormData(form);
161 const statusPesan = document.getElementById('statusPesan');
162 // Mengirim data ke proses_kontak.php di latar belakang menggunakan fetch API
163 fetch('proses_kontak.php', {
164
165 method: 'POST',
166 body: formData
167 })
168 .then(response => response.text())
169 .then(data => {
```

```
index.html X proses_kontak.php koneksi.php
C:\xampp\htdocs> web-getpost > index.html > html > body > div.container > div.table-wrapper > table > tbody#isiTable
2 <html lang="id">
142 <body>
143 <div class="container">
144 <div class="form-wrapper">
157 <script>
159 const form = document.getElementById('formKontak');
160 const formData = new FormData(form);
161 const statusPesan = document.getElementById('statusPesan');
162 // Mengirim data ke proses_kontak.php di latar belakang menggunakan fetch API
163 fetch('proses_kontak.php', {
164
165 method: 'POST',
166 body: formData
167 })
168 .then(response => response.text())
169 .then(data => {
170 // Menampilkan pesan respons yang dikirim oleh file PHP
171 statusPesan.innerHTML = data;
172 statusPesan.style.display = 'block';
173 // Menghilangkan pesan secara otomatis setelah jeda 3 detik
174 setTimeout(() => {
175 statusPesan.style.display = 'none';
176 }, 3000);
177 // Mengosongkan isian formulir setelah data berhasil terkirim
178 form.reset();
179 })
180 .catch(error => {
181 console.error('Error:', error);
182 statusPesan.innerHTML = "Terjadi kesalahan saat mengirim data.";
183 });
184 </script>
```

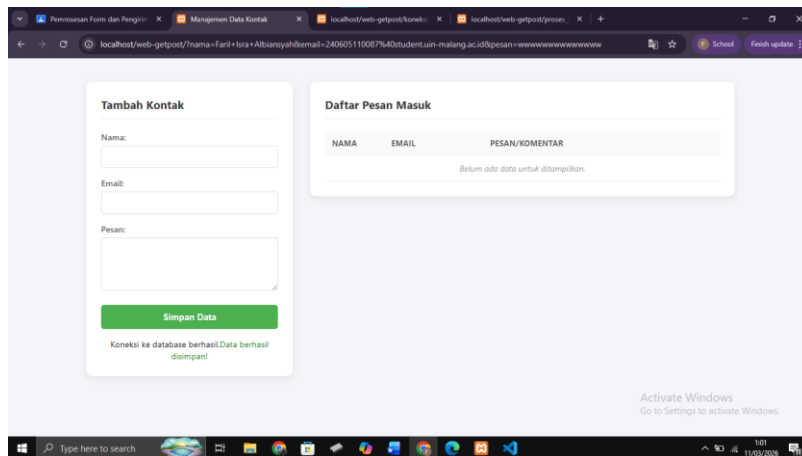


```
index.html x proses_kontak.php koneksi.php
C:\> xampp > htdocs > web-getpost > index.html > html > body > div.container > div.table-wrapper > table > tbody#isiTabel
2 <html lang="id">
142 <body>
143 <div class="container">
144 <div class="form-wrapper">
157 <script>
158     statusPesan.innerHTML = 'Berjasa Keselamatan Saat Mengirim Data.';
159     });
160 </script>
161 </div>
162 <div class="table-wrapper">
163     <h2>Daftar Pesan Masuk</h2>
164     <table>
165         <thead>
166             <tr>
167                 <th class="col-nama">Nama</th>
168                 <th class="col-email">Email</th>
169                 <th class="col-pesan">Pesan/Komentar</th>
170             </tr>
171         </thead>
172         <tbody id="isiTabel">
173             <tr>
174                 <td colspan="3" class="empty-msg">Belum ada data untuk ditampilkan.</td>
175             </tr>
176         </tbody>
177     </table>
178 </div>
179 </div>
180 </body>
181 </html>
```

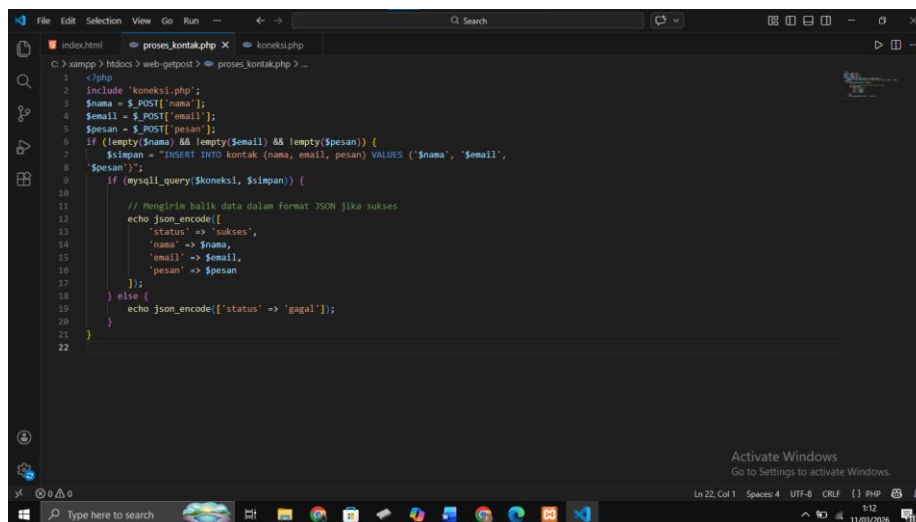
Penggunaan atribut id="isiTabel" pada tag <tbody> berfungsi sebagai penanda unik bagi JavaScript untuk memanipulasi struktur tabel di sisi klien. Dengan adanya identitas ini, skrip dapat mengakses bagian badan tabel secara langsung untuk menambah data tanpa memengaruhi bagian kepala <thead> atau kaki <tfoot> tabel. Selain itu, terdapat baris pesan "Belum ada data untuk ditampilkan" yang disiapkan sebagai tampilan awal (placeholder) sebelum proses penyimpanan data secara asynchronous dilakukan.

Output :

Perubahan utama pada bagian ini adalah penggunaan instruksi echo yang menyertakan tag HTML lengkap dengan gaya visualnya (inline CSS). Teks ini nantinya akan diterima oleh fungsi fetch() pada JavaScript dan langsung dimasukkan ke dalam elemen statusPesan yang telah disiapkan sebelumnya. Dengan cara ini, umpan balik yang diterima pengguna menjadi lebih informatif karena perbedaan status (sukses, gagal, atau peringatan) dibedakan melalui warna teks yang kontras. Maka pesan akan muncul di bawah tombol "Simpan Data" seperti yang ditunjukkan gambar berikut:



Setelah berhasil menampilkan pesan konfirmasi, berikutnya adalah bagaimana membuat tabel di sebelah kanan formulir terisi secara otomatis tanpa perlu memuat ulang halaman. Untuk mewujudkannya, format JSON (JavaScript Object Notation) akan digunakan sebagai jembatan pertukaran data antara PHP dan JavaScript. Langkah pertama adalah memodifikasi file `proses_kontak.php`. Sebagai respon dari server, server akan mengirimkan sebuah paket data terstruktur yang berisi status keberhasilan beserta isi data yang baru saja disimpan.



Fungsi `json_encode()` digunakan untuk mengubah array asosiatif PHP menjadi format JSON yang dapat dikenali secara universal oleh berbagai bahasa pemrograman, termasuk JavaScript. Data yang dikirimkan balik ke browser tidak

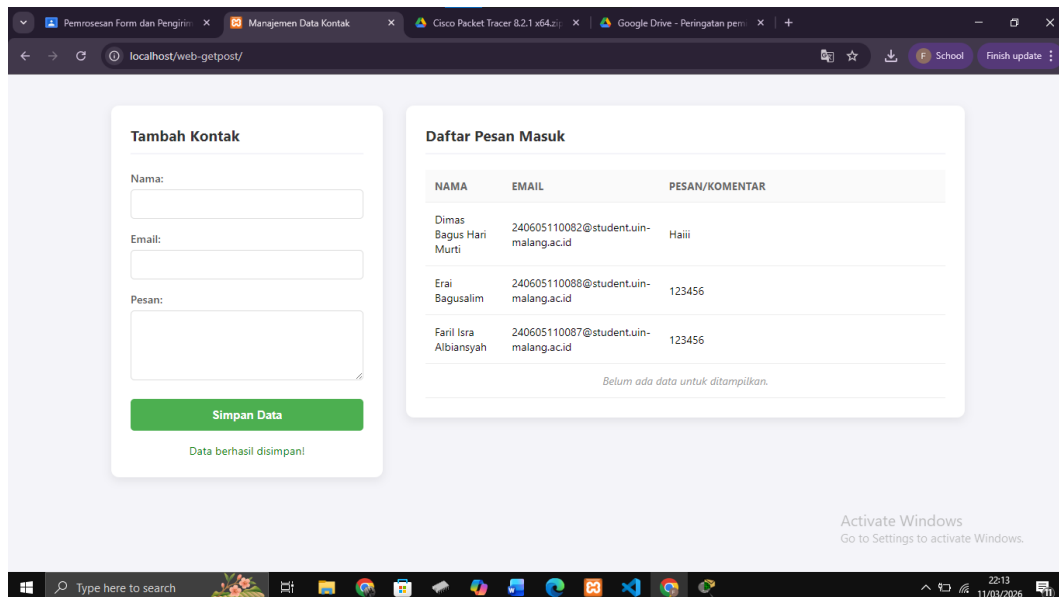
hanya berupa status keberhasilan (status), tetapi juga menyertakan kembali variabel nama, email, dan pesan yang telah divalidasi dan disimpan ke database.

Pada sisi klien, fungsi `fetch()` perlu disesuaikan agar dapat mengolah respons JSON tersebut dan menyisipkannya ke dalam baris tabel secara dinamis menggunakan manipulasi DOM (Document Object Model).

```
1  <script>
2
3      function simpanData() {
4          const form = document.getElementById('formKontak');
5          const formData = new FormData(form);
6          const statusPesan = document.getElementById('statusPesan');
7          const tabel = document.getElementById('isiTabel'); // Pastikan <tbody> memiliki ID
8          ini
9          fetch('proses_kontak.php', {
10             method: 'POST',
11             body: formData
12         })
13         .then(response => response.json()) // Mengubah respons menjadi objek JSON
14         .then(data => {
15             if (data.status === 'sukses') {
16                 // 1. Tampilkan pesan sukses
17                 statusPesan.innerHTML = "<span style='color: green;'>Data berhasil disimpan!</span > ";
18                 // 2. Tambahkan baris baru ke posisi paling atas tabel
19                 const barisBaru = tabel.insertRow(0);
20                 barisBaru.innerHTML = `
21                     <td>${data.nama}</td>
22                     <td>${data.email}</td>
23                     <td>${data.pesan}</td> `;
24                 // 3. Reset form and bersihkan pesan setelah 3 detik
25                 form.reset();
26                 setTimeout(() => { statusPesan.innerHTML = ""; }, 3000);
27             } else {
28                 statusPesan.innerHTML = "<span style='color: red;'>Gagal menyimpan data.</span > ";
29             }
30         });
31     }
32 </script>
```

Dari kode program tersebut terlihat bahwa penggunaan JSON memungkinkan JavaScript untuk menerima data mentah dari server dan menyusunnya kembali ke dalam struktur HTML yang diinginkan. Dengan memanfaatkan metode `insertRow(0)`, data yang baru saja disimpan akan langsung muncul di baris paling atas tabel tanpa mengganggu data yang sudah ada sebelumnya.

Adapun tampilannya seperti pada gambar berikut:



IV. Kesimpulan

Praktikum ini berhasil mensimulasikan mekanisme pengolahan data dari sisi klien (*client-side*) ke sisi server (*server-side*) dengan fokus pada interaktivitas aplikasi web.

1. Metode Pengiriman Data (POST)

- Formulir menggunakan metode **POST** karena lebih aman dan rapi, di mana data dikirim secara tersembunyi tanpa terlihat pada bilah alamat (*address bar*) browser.
- Data yang dikirimkan melalui metode ini ditangkap di sisi server menggunakan variabel superglobal **\$_POST** yang merupakan array asosiatif.

2. Integrasi Database dan Validasi

- Penyimpanan data memerlukan jalur koneksi yang stabil melalui fungsi `mysqli_connect`.
- Pemisahan kode koneksi ke dalam file koneksi.php dilakukan untuk menjaga kerapian kode dan memudahkan pengelolaan (*reusability*).

- Validasi dilakukan di dua sisi: atribut required pada HTML sebagai validasi awal di sisi klien , serta pengecekan fungsi empty() di sisi PHP untuk memastikan integritas data sebelum menjalankan perintah SQL INSERT.

3. Komunikasi Asynchronous dengan AJAX (Fetch API)

- Penggunaan fungsi **fetch()** pada JavaScript memungkinkan pengiriman data ke server di latar belakang tanpa perlu melakukan penyegaran (*refresh*) halaman.
- Teknik ini meningkatkan pengalaman pengguna (*User Experience*) karena umpan balik (seperti pesan sukses atau gagal) muncul secara instan di bawah tombol.

4. Peran Format JSON

- **JSON (JavaScript Object Notation)** berfungsi sebagai jembatan pertukaran data yang ringan dan universal antara PHP dan JavaScript.
- Dengan mengubah respons PHP menjadi JSON melalui json_encode(), JavaScript dapat menangkap data mentah dan menyisipkannya ke dalam struktur tabel secara dinamis melalui manipulasi DOM.